Semana 11: Deep Learning — redes neuronales con PyTorch

Nivel: Intermedio-avanzado | **Prerrequisitos:** Semana 10 (flujo `X`/`y`, entrenamiento/prueba, sobreajuste, métricas, regresión logística), Semana 9 (NumPy, arreglos y `@`), Semana 8 (probabilidad, semilla reproducible), Semana 7 (producto punto, derivada, descenso del gradiente), Semana 6 (clases, `pytest`) y Semana 3 (funciones y bucles) | **Versión:** v1.0 | **Fecha:** 2026-11-02

Objetivos de aprendizaje

1. Describir una **neurona artificial** (`z = w·x + b` más una función de activación) y una **red** como capas de neuronas apiladas.
2. Explicar por qué las **funciones de activación no lineales** (ReLU, sigmoide) son imprescindibles.
3. Explicar el ciclo de entrenamiento: **pasada hacia adelante → pérdida → retropropagación → actualización de pesos**.
4. Usar **tensores** y **autograd** de PyTorch para calcular gradientes automáticamente.
5. Construir y entrenar una red con `nn.Sequential`, una función de pérdida y un optimizador (`Adam`).
6. Evaluar una red con datos de prueba (`model.eval()`, `torch.no_grad()`) y reconocer el sobreajuste.

1. Del modelo lineal a la red neuronal

En la Semana 10 la regresión logística calculaba una puntuación `w·x + b` y la convertía en probabilidad. Eso es, exactamente, **una neurona artificial**.

Analogía: una línea de ensamblaje de expertos. Cada neurona es un inspector que mira varias señales, las pondera según su importancia (los pesos `w`) y decide cuánto "se activa". Una red es una cadena de inspectores en capas: los de la primera capa detectan cosas simples y los siguientes combinan esas señales en ideas más complejas.

```
entradas x  →  [capa oculta: varias neuronas]  →  [capa de salida]  →  predicción
```

En **Deep Learning** ("aprendizaje profundo") la red tiene varias capas; "profundo" se refiere a esa profundidad.

Una neurona en NumPy

```
import numpy as np

def relu(z):      return np.maximum(0, z)     # deja pasar lo positivo, anula lo negativo
def sigmoide(z):  return 1 / (1 + np.exp(-z)) # comprime cualquier número a (0, 1)

x = np.array([1.0, 2.0])     # entradas (características)
w = np.array([0.5, -0.25])   # pesos
b = 0.1                      # sesgo

z = x @ w + b                # producto punto (Semana 7) + sesgo
print(z, relu(z), sigmoide(z))   # 0.1  0.1  0.52497918747894
```

¿Por qué una activación no lineal?

Si apilas capas que solo hacen `w·x + b`, el resultado sigue siendo una única operación lineal: mil capas lineales equivalen a una. Con una **activación no lineal** entre capas, la red puede dibujar fronteras curvas y aprender patrones que una recta no puede. El ejemplo clásico es **XOR** ("uno u otro, pero no ambos"): ninguna recta separa sus clases, y por eso la regresión logística se queda en 50 % de exactitud (verificado, sección 7).

Activación	Fórmula	Uso típico
ReLU	`max(0, z)`	Capas ocultas (opción por defecto)
Sigmoide	`1 / (1 + e^-z)`	Salida de una probabilidad (2 clases)
Softmax	exponenciales normalizadas	Salida de probabilidades (varias clases)

2. Cómo aprende una red

Es el descenso del gradiente de la Semana 7, repetido a gran escala:

1. **Pasada hacia adelante (*forward*):** los datos recorren la red y producen una predicción.
2. **Pérdida (*loss*):** un número que mide qué tan mal predijo (como el MSE de la Semana 10; en clasificación se usa la **entropía cruzada**).
3. **Retropropagación (*backpropagation*):** calcula cuánto contribuyó cada peso al error, aplicando la **regla de la cadena** de capa en capa hacia atrás. Es la derivada de la Semana 7 en muchas dimensiones: el **gradiente**.
4. **Actualización:** cada peso se mueve un pequeño paso contra su gradiente: `w ← w − tasa · gradiente`.

Una pasada completa por todos los datos de entrenamiento se llama **época** (*epoch*). Se repite durante muchas épocas. Los datos grandes se procesan en **lotes** (*batches*) pequeños para no cargarlos todos a la vez.

> **Jerga traducida:** "retropropagación" = "repartir la culpa del error entre todos los pesos, de atrás hacia adelante". "Tasa de aprendizaje" = "qué tan grande es cada paso". Si es muy grande, el descenso rebota; si es muy chica, tarda una eternidad.

3. PyTorch: tensores y autograd

PyTorch es la librería de Deep Learning más usada en investigación e industria. Sus bloques:

- **Tensor:** como un `ndarray` de NumPy (Semana 9), pero con soporte de GPU y de derivadas automáticas.
- **Autograd:** registra las operaciones y calcula los gradientes por ti con `.backward()`.

```
import torch

w = torch.tensor(3.0, requires_grad=True)   # "sigue la pista de las derivadas de w"
x, y_real = torch.tensor(2.0), torch.tensor(10.0)

error = (w * x - y_real) ** 2               # (3·2 − 10)² = 16
error.backward()                            # calcula d(error)/dw

print(error.item(), w.grad.item())          # 16.0  -16.0
```

Comprobación a mano con la Semana 7: `d/dw (w·x − y)² = 2·(w·x − y)·x = 2·(−4)·2 = −16`. Autograd acierta sin que escribas la derivada. Como el gradiente es negativo, aumentar `w` reduce el error.

4. Construir una red con `nn.Module`

`torch.nn` ofrece capas listas (POO de la Semana 6: cada capa es un objeto):

- `nn.Linear(entradas, salidas)`: una capa de neuronas (`w·x + b`).
- `nn.ReLU()`: la activación.
- `nn.Sequential(...)`: encadena capas en orden.

```
from torch import nn

red = nn.Sequential(
    nn.Linear(2, 4),   # 2 entradas → 4 neuronas ocultas
    nn.ReLU(),
    nn.Linear(4, 1),   # 4 → 1 salida (una puntuación, "logit")
)
```

El bucle de entrenamiento (siempre el mismo)

```
perdida_fn  = nn.BCEWithLogitsLoss()                 # sigmoide + entropía cruzada
binaria
optimizador = torch.optim.Adam(red.parameters(), lr=0.05)

for epoca in range(2000):
    optimizador.zero_grad()              # 1. borrar gradientes del paso anterior
    perdida = perdida_fn(red(X), y)      # 2. forward + pérdida
    perdida.backward()                   # 3. retropropagación
    optimizador.step()                   # 4. actualizar los pesos
```

> **Error clásico:** olvidar `zero_grad()`. PyTorch *acumula* gradientes; sin borrarlos, cada paso usa gradientes viejos mezclados con los nuevos.

5. Evaluar: modo evaluación y sin gradientes

Para predecir con datos nuevos:

```
red.eval()                       # apaga comportamientos de entrenamiento (p. ej. Dropout)
with torch.no_grad():            # no registrar operaciones: más rápido y menos memoria
    probabilidades = torch.sigmoid(red(X_nuevos))
```

Las reglas de la Semana 10 siguen vigentes: separar entrenamiento y prueba, escalar solo con el entrenamiento y desconfiar de una nota excelente en entrenamiento y mediocre en prueba (**sobreajuste**). Las redes tienen mucha capacidad y sobreajustan con facilidad: se combaten con más datos, redes más pequeñas, *dropout* (apagar neuronas al azar al entrenar), regularización y detención temprana (parar cuando la pérdida de validación deja de mejorar).

6. Por qué esto importa para IA/ML

- **Todo el ecosistema moderno está construido sobre este ciclo.** Visión por computador, traducción y los LLM de la Semana 12 se entrenan con *forward → pérdida → backward → step*; solo cambian el tamaño y la arquitectura.
- **Una red grande es la neurona de la Semana 7 apilada millones de veces.** Lo que aprendiste (producto punto, derivada, gradiente) escala sin conceptos nuevos.
- **Las redes brillan con datos no estructurados** (imágenes, texto, audio). Con datos tabulares pequeños, los modelos clásicos de la Semana 10 suelen igualar o ganar: se prueba siempre primero una línea base simple.
- **La disciplina de evaluación es idéntica:** división entrenamiento/prueba, métricas, sin fuga de datos.

7. Ejemplo práctico: una red que aprende XOR

Problema: XOR devuelve 1 cuando las dos entradas son distintas. Una regresión logística no puede resolverlo; una red con una capa oculta sí.

Pseudocódigo

```
datos: (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→0
crear red: 2 entradas → 4 neuronas ocultas (ReLU) → 1 salida
repetir 2000 veces:
    borrar gradientes
    predecir los 4 casos y calcular la pérdida
    retropropagar
    actualizar los pesos
convertir las salidas en 0/1 (probabilidad > 0.5) y comparar
```

Python

```
# red_xor.py
"""Una red neuronal mínima en PyTorch que aprende XOR."""
import torch
from torch import nn


def crear_datos():
    X = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
    y = torch.tensor([[0.], [1.], [1.], [0.]])
    return X, y


def crear_red():
    # 2 entradas -> 4 neuronas ocultas (ReLU) -> 1 salida (puntuación, "logit")
    return nn.Sequential(nn.Linear(2, 4), nn.ReLU(), nn.Linear(4, 1))


def entrenar(modelo, X, y, epocas=2000, tasa=0.05):
    perdida_fn = nn.BCEWithLogitsLoss()           # sigmoide + entropía cruzada,
numéricamente estable
    optimizador = torch.optim.Adam(modelo.parameters(), lr=tasa)
    for _ in range(epocas):
        optimizador.zero_grad()                   # borrar gradientes del paso anterior
        perdida = perdida_fn(modelo(X), y)        # 1) pasada hacia adelante + error
        perdida.backward()                        # 2) retropropagación: calcula
gradientes
        optimizador.step()                        # 3) actualizar pesos
    return perdida.item()


def predecir(modelo, X):
    modelo.eval()
    with torch.no_grad():                         # no hace falta autograd al predecir
        return (torch.sigmoid(modelo(X)) > 0.5).int().flatten().tolist()


if __name__ == "__main__":
    torch.manual_seed(0)                          # semilla (Semana 8): resultado
reproducible
    X, y = crear_datos()
    modelo = crear_red()
    print("pérdida final:", round(entrenar(modelo, X, y), 4))
    print("predicciones:", predecir(modelo, X))
```

Resultado verificado por ejecución: `pérdida final: 0.0001` y `predicciones: [0, 1, 1, 0]`. En cambio, `LogisticRegression` sobre los mismos 4 puntos obtiene una exactitud de **0.5**: el límite de un modelo lineal.

Segundo ejemplo: Iris con una red (mismo flujo que la Semana 10)

```
# iris_torch.py
import torch
from torch import nn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler


def preparar_datos():
    X, y = load_iris(return_X_y=True)
    X_ent, X_prueba, y_ent, y_prueba = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y)
    escalador = StandardScaler().fit(X_ent)            # solo con entrenamiento (sin fuga)
    a_tensor = lambda a, t: torch.tensor(a, dtype=t)
    return (a_tensor(escalador.transform(X_ent), torch.float32), a_tensor(y_ent,
torch.long),
            a_tensor(escalador.transform(X_prueba), torch.float32), a_tensor(y_prueba,
torch.long))


def entrenar(X, y, epocas=200, tasa=0.01):
    torch.manual_seed(42)
    modelo = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 3))
    perdida_fn = nn.CrossEntropyLoss()                 # espera "logits" y etiquetas
enteras
    optimizador = torch.optim.Adam(modelo.parameters(), lr=tasa)
    for _ in range(epocas):
        optimizador.zero_grad()
        perdida_fn(modelo(X), y).backward()
        optimizador.step()
    return modelo


def exactitud(modelo, X, y):
    modelo.eval()
    with torch.no_grad():
        return (modelo(X).argmax(dim=1) == y).float().mean().item()
```

Resultado verificado: exactitud 0.983 en entrenamiento y 1.0 en prueba (30 flores; con tan pocos ejemplos, una sola división es poco concluyente, como se advirtió en la Semana 10). Nota: `CrossEntropyLoss` ya incluye el softmax, por eso la última capa **no** lleva activación.

Pruebas con pytest (Semana 6)

```
# test_red.py
import torch
from red_xor import crear_datos, crear_red, entrenar, predecir
import iris_torch


def test_autograd_calcula_la_derivada():
    w = torch.tensor(3.0, requires_grad=True)
    ((w * 2.0 - 10.0) ** 2).backward()
    assert w.grad.item() == -16.0


def test_forma_de_la_red():
    assert crear_red()(crear_datos()[0]).shape == (4, 1)


def test_red_aprende_xor():
    torch.manual_seed(0)
    X, y = crear_datos()
    modelo = crear_red()
    entrenar(modelo, X, y)
    assert predecir(modelo, X) == [0, 1, 1, 0]


def test_perdida_baja_al_entrenar():
    torch.manual_seed(0)
    X, y = crear_datos()
    modelo = crear_red()
    antes = torch.nn.BCEWithLogitsLoss()(modelo(X), y).item()
    despues = entrenar(modelo, X, y, epocas=500)
    assert despues < antes


def test_iris_supera_90_por_ciento():
    X_ent, y_ent, X_p, y_p = iris_torch.preparar_datos()
    modelo = iris_torch.entrenar(X_ent, y_ent)
    assert iris_torch.exactitud(modelo, X_p, y_p) > 0.9
```

Las 5 pruebas pasan (verificado con PyTorch 2.15, scikit-learn 1.9 y NumPy 2.5).

8. Notas de vigencia técnica (2026)

- **PyTorch es el estándar de facto** en investigación y gran parte de la industria. **TensorFlow** sigue existiendo, pero el ecosistema (modelos preentrenados, Hugging Face, LLM) gira alrededor de PyTorch. **Keras 3** es multi-backend (PyTorch, JAX, TensorFlow) y es una alternativa de alto nivel; **JAX** se usa mucho en investigación a gran escala.
- **Tutoriales antiguos:** `torch.autograd.Variable` está obsoleto (los tensores ya hacen autograd); `TensorFlow 1.x` con `Session` ya no se usa.
- **Optimizador por defecto:** `Adam` / `AdamW` funcionan bien sin mucho ajuste; SGD con *momentum* sigue vigente en visión.
- **Pérdidas con logits:** prefiere `BCEWithLogitsLoss` y `CrossEntropyLoss` (más estables) en lugar de añadir `sigmoid`/`softmax` a mano antes de la pérdida.
- **GPU:** con `torch.device("cuda" if torch.cuda.is_available() else "cpu")` mueves modelo y datos (`.to(dispositivo)`); los ejemplos de esta clase corren bien en CPU. `torch.compile` acelera modelos grandes.
- **Datos grandes:** se usan `Dataset` y `DataLoader` para lotes (no se necesitan para los ejemplos de hoy).
- **Reproducibilidad:** fija `torch.manual_seed(...)` y las versiones (`requirements.txt`); en GPU algunos resultados pueden variar levemente.
- **Para datos tabulares**, el *gradient boosting* de la Semana 10 sigue siendo, con frecuencia, la mejor opción.

9. Errores comunes de principiante

- **Olvidar `optimizador.zero_grad()`** y acumular gradientes.
- **Aplicar `softmax`/`sigmoid` antes de `CrossEntropyLoss`/`BCEWithLogitsLoss`**, que ya lo incluyen.
- **Etiquetas con el tipo equivocado:** `CrossEntropyLoss` pide enteros (`torch.long`); `BCEWithLogitsLoss` pide decimales (`float`).
- **No escalar las entradas** o escalar usando también los datos de prueba (fuga de datos).
- **Evaluar con los datos de entrenamiento** o dejar el modelo en modo entrenamiento al predecir (sin `eval()`/`no_grad()`).
- **Tasa de aprendizaje inadecuada:** pérdida que explota o que no baja.
- **Usar una red cuando basta una regresión logística:** compara siempre con la línea base.
- **Creer que "más capas" siempre mejora:** más capacidad también significa más riesgo de sobreajuste.

10. Ejercicio propuesto

Escribe `red_vinos.py` con `cargar_datos()`, `crear_red()`, `entrenar()` y `exactitud()` siguiendo el diseño de `iris_torch.py`, usando `load_wine` de scikit-learn (13 características, 3 clases).

Entrena una red con una capa oculta y compara su exactitud en prueba con la `LogisticRegression` de la Semana 10. Muestra la exactitud en entrenamiento y en prueba y deja como comentario si hay sobreajuste. Añade pruebas con `pytest` y versiona con Git.

Pista: ¿cuántas entradas y salidas tiene la primera y la última `nn.Linear`?

11. Autoevaluación

1. ¿Qué dos operaciones hace una neurona artificial? — Una combinación lineal `w·x + b` y luego una función de activación.
2. ¿Por qué una red necesita activaciones no lineales? — Sin ellas, apilar capas equivale a una sola operación lineal y no podría aprender patrones como XOR.
3. Ordena: actualizar pesos, forward, retropropagación, pérdida. — Forward → pérdida → retropropagación → actualizar pesos.
4. ¿Qué es una época? — Una pasada completa por todos los datos de entrenamiento.
5. ¿Qué hace `.backward()` y dónde queda el resultado? — Calcula los gradientes por autograd y los guarda en `.grad` de cada parámetro.
6. ¿Por qué se llama `optimizador.zero_grad()` en cada paso? — PyTorch acumula gradientes; hay que borrarlos para no mezclar pasos distintos.
7. ¿Para qué sirven `model.eval()` y `torch.no_grad()`? — Predecir en modo evaluación y sin registrar gradientes (más rápido, menos memoria).
8. Una red tiene 99 % de exactitud en entrenamiento y 70 % en prueba. ¿Qué pasa? — Sobreajuste: memorizó el entrenamiento; se reduce con más datos, red más pequeña, dropout o detención temprana.
9. V/F: "Con datos tabulares pequeños una red siempre supera a un modelo clásico." — Falso: suelen igualarse o ganar los clásicos; se compara con una línea base.

12. Verificación de dependencias

Depende de la Semana 10 (flujo `X`/`y`, `train_test_split`, `StandardScaler`, sobreajuste, regresión logística como una neurona, métricas, `fit`/`predict`), la Semana 9 (arreglos NumPy, `@`, base de los tensores), la Semana 8 (sigmoide y probabilidades, semilla reproducible), la Semana 7 (producto punto, derivada, regla de la cadena y descenso del gradiente, base de la retropropagación), la Semana 6 (clases/objetos como `nn.Module`, `pytest`, Git en el ejercicio) y la Semana 3 (funciones y bucles). PyTorch se introduce desde cero; no se requiere nada de la Semana 12 (LLMs, RAG, agentes). Es prerrequisito directo del Módulo 12, que usa redes profundas (Transformers) preentrenadas.